

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION
TECHNOLOGY



SOICT

FINAL PROJECT REPORT

Topic: **Parallelization Methods for CNN-Based Computer Vision Inference**

Course: **Parallel and Distributed Programming**

Course code: IT4130E

Lecturer: **Nguyen Tuan Dung**

Group members: **Luu Hieu An - 202400093**

Nguyen Phu An - 20235466

Pham Chi Bang - 20235477

Nguyen Thanh Lam - 20235518

Hanoi, 2026

Contents

Abstract	1
1 Introduction	2
2 Problem Description	3
2.1 Problem Context	3
2.2 YOLO Video People Counting	3
2.3 Temporal-Spatial Task Definition	4
2.4 Post-Processing Challenge	5
2.5 Parallel-Programming Focus	5
2.6 Correctness Targets	5
3 Parallelization Methods	7
3.1 Task Parallel YOLO Inference	7
3.1.1 Temporal-Spatial Decomposition	7
3.1.2 1D Mapping and Static Scheduling	7
3.1.3 Communication and Merging	7
3.1.4 YOLO Algorithm Sketch	9
3.2 Weighted Process Placement	9
3.3 Metric Collection	10
3.4 Load Balancing and Metrics	10
4 Experimental Setup	11
4.1 Physical Cluster	11
4.2 Software and Data	11
4.3 Metrics	11
5 Results and Discussion	12
5.1 Detector Accuracy Under Tile Splitting	12
5.2 Input-Size Selection	12
5.3 Granularity and Load Balance	13
5.4 Speedup and Efficiency	14
5.5 Weighted Static Placement	15
5.6 Non-Blocking Gather Evaluation	15
5.7 YOLO Parallel Correctness	16
5.8 Summary of Findings	16
6 Conclusion	17

Abstract

Video people counting is an important computer vision problem for crowd monitoring, traffic analysis, safety management, and real-time camera systems, but applying a CNN detector to every frame of a video is computationally expensive. This report presents *MPI-Based YOLO People Counting*, a C++17/OpenMPI pipeline that accelerates pretrained YOLO inference by treating video analysis as a task-parallel workload. The input video is decomposed temporally into frames and spatially into image tiles, so each frame-tile pair becomes an independent MPI task assigned to a rank for local YOLO inference. After inference, rank 0 gathers the detected boxes, remaps tile-local coordinates back to the original frame, removes duplicate detections near tile boundaries, applies global non-maximum suppression, and produces frame-level people counts. The report studies this pipeline as a distributed-memory parallel program, focusing on decomposition, task mapping, communication, load balancing, and the distinction between preserving the serial pipeline output and measuring detector accuracy against ground truth. Overall, the work shows how a practical video people-counting application can be expressed as a task-level MPI problem while exposing the main trade-offs between parallel speedup, communication overhead, tile granularity, and heterogeneous machine capacity.

1 Introduction

Video people counting is a practical computer vision problem with applications in crowd monitoring, traffic analysis, safety management, and camera-based automation. A modern detector such as YOLO can identify people in individual images, but applying it repeatedly to every frame of a video quickly becomes computationally expensive [6, 7]. This makes the problem useful for a parallel-programming study: it is realistic enough to require a full inference pipeline, yet structured enough to expose clear opportunities for decomposition, scheduling, communication analysis, and speedup measurement.

This project focuses on inference rather than model training. The detector itself is a pretrained YOLO model, while the parallel-programming work is the C++17/OpenMPI runtime built around it [5, 3]. The system decomposes a video temporally into frames and spatially into image tiles, then treats each frame-tile pair as an MPI task. Each rank runs YOLO on its assigned tasks through a long-lived local worker process. After inference, rank 0 gathers detections, converts tile-local bounding boxes back to full-frame coordinates, removes duplicate detections near tile boundaries, computes frame-level people counts, and writes the experiment artifacts.

The report evaluates the implementation as a distributed-memory parallel program and separates parallel correctness from detector accuracy. Parallel correctness asks whether the MPI implementation preserves the output of the serial version of the same pipeline, while detector accuracy asks whether YOLO’s predicted counts match human ground truth from MOT17-derived data [4]. This distinction is important because the detector may undercount people in crowded scenes even when the MPI program is correct. The experiments therefore measure correctness, input-size suitability, scheduling behavior, granularity, speedup, and heterogeneous rank placement.

The main contributions and insights of this technical report are:

1. It formulates YOLO video people counting as a task-parallel MPI workload over frames and image tiles.
2. It presents an OpenMPI implementation that combines static block-cyclic task mapping, rank-local YOLO workers, centralized detection merging, duplicate removal, and metric collection.
3. It verifies that the MPI pipeline preserves serial output in the correctness experiment, while also showing that detector accuracy against ground truth is a separate limitation.
4. It analyzes speedup and remaining bottlenecks, especially communication overhead, tile granularity, master-side post-processing, and unequal machine capacity.

2 Problem Description

2.1 Problem Context

Video people counting asks a system to estimate how many people appear in each frame of a video. This problem is useful in crowd monitoring, classroom analytics, transportation observation, safety systems, and other camera-based applications where the number of people changes over time. In this report, people counting is implemented as an object-detection problem: a detector predicts person bounding boxes, and the final count for a frame is the number of accepted person boxes after post-processing.

The computational challenge comes from repeated CNN inference. A single YOLO inference may be manageable, but a video contains hundreds or thousands of frames, and each frame may be split into several regions to expose more parallel work. The total workload therefore grows with both the number of frames and the number of tiles per frame. This makes the application a good parallel-programming problem: it contains many independent inference calls, but the final counting step still requires careful global merging.

The project focuses on inference rather than training. The YOLO model is fixed and pretrained; the report does not modify model weights or propose a new detector. The technical problem is to build a distributed-memory runtime that executes the same inference pipeline faster on multiple MPI ranks while preserving the output of the corresponding serial pipeline.

2.2 YOLO Video People Counting

The input is a finite video sequence:

$$V = \{F_1, F_2, \dots, F_N\}. \quad (2.1)$$

For each frame F_i , the detector produces a set of candidate person detections. Each detection contains a bounding box, a confidence score, and a class label. After filtering and duplicate removal, the target output is the number of final person boxes:

$$\text{count}(F_i) = |\{b \mid b \text{ is a final person bounding box in } F_i\}|. \quad (2.2)$$

The full output of the pipeline is therefore a sequence of frame-level counts:

$$C = \{\text{count}(F_1), \text{count}(F_2), \dots, \text{count}(F_N)\}. \quad (2.3)$$

The detector backend is a pretrained YOLO11n model executed through a local Python worker process [7]. YOLO11 follows the usual one-stage detector structure with a feature-extraction backbone, a feature-fusion neck, and multi-scale detection heads, as shown in Figure 2.1. In this project, YOLO is treated as a detector backend. The MPI program does not split individual neural-network layers or kernels; instead, it distributes repeated YOLO calls across independent video regions.

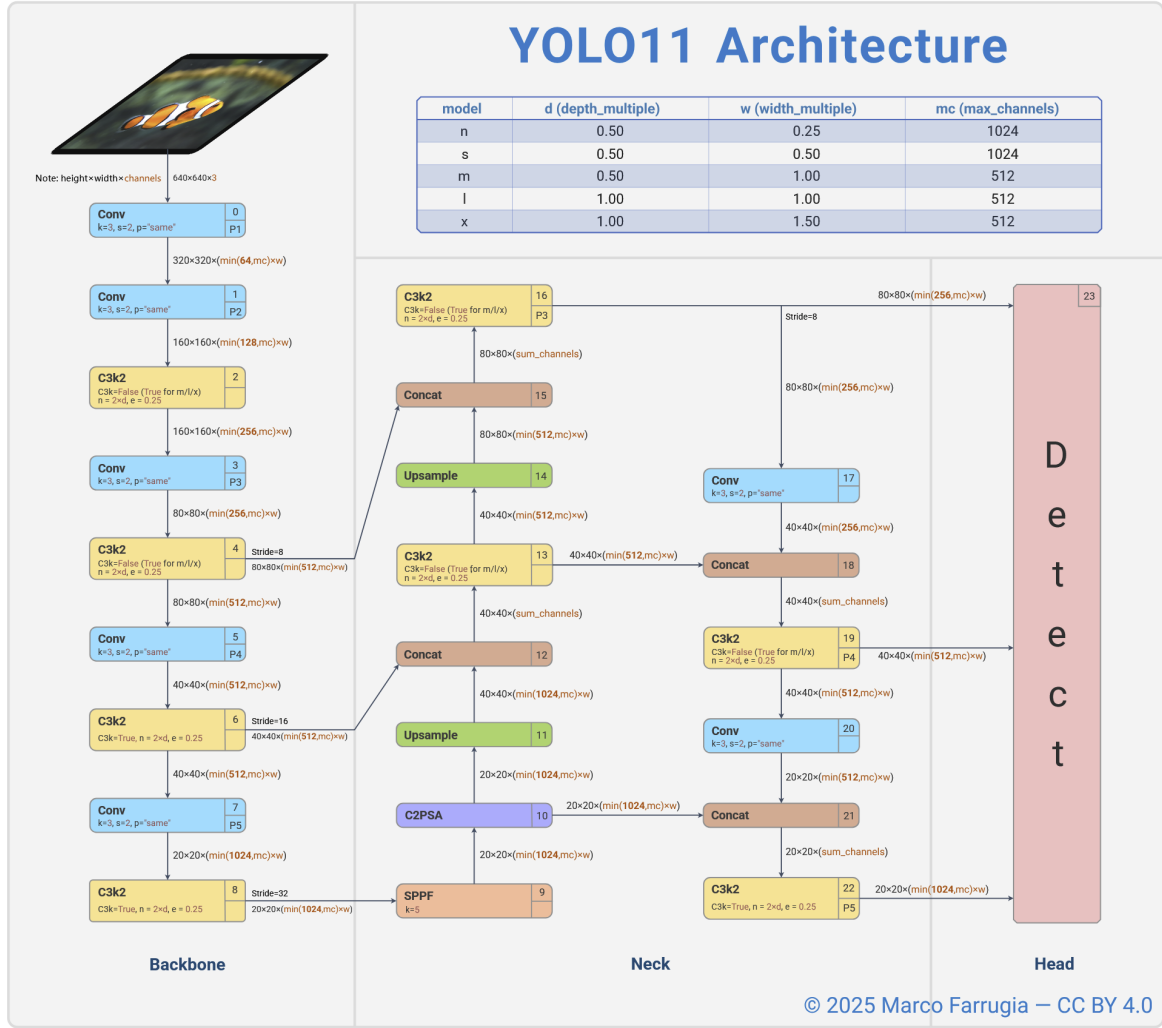


Figure 2.1: YOLO11 object-detection architecture used as the detector backend in this project. Source: Farrugia, CC BY 4.0, Zenodo DOI 10.5281/zenodo.17508018 [1].

2.3 Temporal-Spatial Task Definition

Frame-only parallelism is possible because frames are naturally ordered but mostly independent for this counting task. However, if only complete frames are distributed, the number of tasks may be too small to keep many MPI ranks busy, especially for short videos or small experiments. The project therefore uses a hybrid decomposition: each video is split by time into frames, and each frame is split by space into image tiles.

For a tile grid with R rows and C columns, frame F_i is decomposed into:

$$F_i \rightarrow \{T_{i,1}, T_{i,2}, \dots, T_{i,R \times C}\}. \quad (2.4)$$

A parallel task is one YOLO inference on one frame-tile pair:

$$task(i, j) = T_{i,j}, \quad |\mathcal{T}| = N \times R \times C. \quad (2.5)$$

This definition turns video inference into a task-parallel workload. Larger values of R and C create more tasks, which can improve utilization and load balancing. At the same time, finer tiling can reduce detector context, cut people near tile boundaries, create duplicate boxes, and increase the amount of data that must be gathered by rank 0. The problem is therefore not only to create more tasks, but also to choose a granularity that exposes parallelism without making merging and communication too expensive.

2.4 Post-Processing Challenge

The output of each tile is local to that tile. If a detected box has local coordinates (x_1, y_1, x_2, y_2) , these coordinates must be shifted back into the coordinate system of the original frame before the final count can be computed. For a tile whose top-left corner is (x_s, y_s) , the global coordinates are:

$$x^{global} = x^{local} + x_s, \quad y^{global} = y^{local} + y_s. \quad (2.6)$$

This coordinate remapping is necessary but not sufficient. A person standing near a tile boundary may be detected in more than one tile. If all tile detections were counted directly, the final frame count could be too high. Therefore, rank 0 must merge detections from all tiles of the same frame, apply tile ownership filtering, and run global non-maximum suppression before producing the final count:

$$B_i^{final} = NMS(B_i^{raw}, \theta), \quad count(F_i) = |B_i^{final}|. \quad (2.7)$$

This post-processing step is the main reason the problem is not purely embarrassingly parallel. Detector calls can be distributed across ranks, but final counting requires global information about all detections belonging to the same frame.

2.5 Parallel-Programming Focus

The central parallel-programming question is how to execute the frame-tile inference tasks across multiple MPI ranks while preserving the serial pipeline output and measuring the cost of distribution. The implementation uses task parallelism, static task mapping, centralized result gathering, and weighted process placement on a heterogeneous three-machine cluster.

The YOLO pipeline exercises the following parallel-programming concepts:

Table 2.1: Parallel-programming concepts in the YOLO pipeline

Concept	YOLO MPI implementation
Parallelism level	Task parallelism across repeated YOLO inference calls
Decomposition	Hybrid temporal-spatial decomposition into frame-tile tasks
Mapping	1D flattened task list with static block-cyclic ownership
Communication	Rank-local inference followed by centralized detection and metric gathering
Load balancing	Tile granularity and weighted process placement
Correctness target	MPI frame counts match serial frame counts after the same post-processing pipeline

2.6 Correctness Targets

The report uses two different notions of correctness. The first is parallel correctness. The serial YOLO baseline processes the same frame-tile task list without MPI distribution and applies the same post-processing pipeline. The MPI implementation is considered correct when it produces the same final people count as this serial baseline for each tested frame:

$$\Delta_i = |count_{parallel}(F_i) - count_{serial}(F_i)|. \quad (2.8)$$

The correctness experiment reports mismatched frames, maximum count error, and mean count error. A zero mismatch means the parallelization has preserved the serial pipeline output for the tested configuration.

The second notion is detector accuracy. Detector accuracy compares YOLO's predicted counts against human ground-truth counts from MOT17-derived data. This is a separate question because a model can undercount people in crowded frames even if the MPI implementation is perfectly consistent with the serial pipeline. The report therefore evaluates model accuracy and parallel

correctness separately: one measures the quality of the detector, while the other measures whether the distributed implementation changes the pipeline result.

3 Parallelization Methods

3.1 Task Parallel YOLO Inference

The YOLO pipeline uses task-level parallelism over independent frame-tile inference tasks. The implementation is centered on `src/yolo_mpi_cpp.cpp` in the repository. Each MPI rank runs the same C++ program, and rank-local detector work is delegated to a long-lived Python YOLO worker.

3.1.1 Temporal-Spatial Decomposition

The input video is first decomposed by time into frames and then by space into image tiles. If the video has N frames and each frame is split into an $R \times C$ grid, the total number of tasks is NRC . Figure 3.1 compares the original frame with 2x2 and 5x4 tiling. The 2x2 grid keeps detector accuracy close to the original-frame baseline because it introduces fewer artificial tile borders, so fewer people are cut by tile edges before the final merge stage. Larger grids create more tasks and can improve scheduling flexibility, but they increase duplicate detections near tile borders and increase post-processing work.

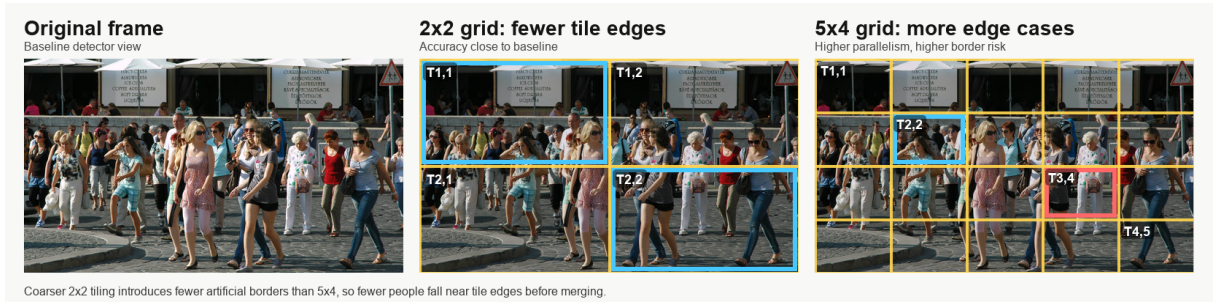


Figure 3.1: Hybrid temporal-spatial decomposition for YOLO video inference. The public-domain crowd frame is from Wikimedia Commons [2]; the overlays show why coarse 2x2 tiling preserves accuracy better than finer 5x4 tiling while still creating independent frame-tile tasks.

3.1.2 1D Mapping and Static Scheduling

All frame-tile tasks are flattened into a one-dimensional list. For task index i , chunk size k , and P ranks, static block-cyclic scheduling assigns:

$$\text{owner}(\text{task}_i) = \left\lfloor \frac{i}{k} \right\rfloor \bmod P. \quad (3.1)$$

Static scheduling has low dispatch overhead because ranks can determine their own tasks locally. The measured repository results show that this advantage matters for the 600-frame 5x4 tiled YOLO workload.

3.1.3 Communication and Merging

After each rank finishes its assigned frame-tile tasks, it sends two types of data to rank 0: detected bounding boxes and timing metrics. The communication pattern is centralized. Worker ranks compute locally, then rank 0 gathers the local result payloads and performs the final merge. The final optimization branch uses non-blocking result collection for this gather stage. It does not change YOLO inference, task mapping, or merge semantics; it only changes how rank-local payloads are transferred to rank 0.

The non-blocking gather is implemented in two phases. First, each worker sends its serialized payload size $n_r = |S_r|$ to rank 0 using non-blocking point-to-point communication. Rank 0 posts

matching receives, waits for all sizes, and computes displacements:

$$d_r = \sum_{j=0}^{r-1} n_j. \quad (3.2)$$

Second, rank 0 allocates the final receive buffer and posts non-blocking receives for all payloads S_r , while worker ranks send their serialized detections and metrics. The implementation uses `MPI_Isend`, `MPI_Irecv`, and `MPI_Waitall`, so messages from different ranks can be in flight at the same time. This reduces result-collection overhead compared with a strictly blocking gather. However, inference and communication are not overlapped: ranks first finish their assigned YOLO tasks, then send results.

For each detected person box, the worker stores:

$$(frame_id, tile_id, x_1, y_1, x_2, y_2, score, class). \quad (3.3)$$

The box coordinates are still relative to the local tile. Rank 0 first converts them back to original-frame coordinates:

$$x^{global} = x^{local} + x_s, \quad y^{global} = y^{local} + y_s, \quad (3.4)$$

where (x_s, y_s) is the top-left coordinate of the tile inside the original frame.

For each frame F_i , rank 0 merges all detections from the $R \times C$ tiles:

$$B_i^{raw} = \bigcup_{j=1}^{RC} B_{i,j}. \quad (3.5)$$

Since a person near a tile boundary may appear in more than one tile, duplicate boxes can occur. Rank 0 therefore applies tile-owner filtering followed by global non-maximum suppression:

$$B_i^{final} = NMS(B_i^{raw}, \theta). \quad (3.6)$$

The final people count for frame F_i is:

$$count(F_i) = |B_i^{final}|. \quad (3.7)$$

This centralized merge is necessary because each rank only sees its own assigned tiles, while duplicate removal requires all detections from the same frame.

3.1.4 YOLO Algorithm Sketch

Algorithm 1 Static MPI YOLO people-counting pipeline

- 1: Input video contains N frames; each frame is split into an $R \times C$ grid.
- 2: Build task list $\mathcal{T}$, where each task is $(frame_id, tile_id)$.
- 3: Flatten $\mathcal{T}$ into a 1D list and assign task index q using

$$owner(q) = \left\lfloor \frac{q}{K} \right\rfloor \bmod P,$$

where K is chunk size and P is the total number of MPI ranks. The value $owner(q)$ is a global MPI rank id, not a physical machine id.

- 4: **for** each MPI rank r in parallel **do**
 - 5: Select all tasks whose owner is r .
 - 6: **for** each selected task (i, j) **do**
 - 7: Crop tile j from frame F_i .
 - 8: Run YOLO inference on the tile.
 - 9: Keep only **person** detections.
 - 10: Store local boxes and timing metrics.
 - 11: **end for**
 - 12: **end for**
 - 13: Gather serialized detections and rank metrics to rank 0 using two-phase non-blocking point-to-point communication.
 - 14: **if** rank is 0 **then**
 - 15: **for** each frame F_i **do**
 - 16: Convert tile-local boxes to global frame coordinates.
 - 17: Merge boxes from all tiles of frame F_i .
 - 18: Apply tile-owner filtering and global NMS.
 - 19: Compute $count(F_i) = |B_i^{final}|$.
 - 20: **end for**
 - 21: Write per-frame counts and runtime metrics to CSV files.
 - 22: **end if**
-

3.2 Weighted Process Placement

The cluster is heterogeneous because the three machines do not have the same CPU throughput. If every machine receives the same number of MPI ranks, the faster machines may finish early and wait for the slower machine. This creates idle time and increases the total runtime.

Weighted process placement keeps the same static block-cyclic task mapping, but changes how many MPI ranks are launched on each physical machine. A stronger machine receives more ranks, so more global rank ids are mapped to that machine. A weaker machine receives fewer ranks, so fewer global rank ids are mapped to it. Because tasks are assigned to ranks, this rank placement indirectly changes each machine's share of the work. The report compares uniform 4/4/4 placement against weighted placements such as 3/6/3 and 4/6/2. Here, 4/6/2 means the master runs 4 MPI processes, node1 runs 6 MPI processes, and node2 runs 2 MPI processes.

The scheduling formula does not change:

$$owner(q) = \left\lfloor \frac{q}{K} \right\rfloor \bmod P. \quad (3.8)$$

In this equation, $P = 12$ for all three-machine runs, regardless of whether the placement is 4/4/4, 3/6/3, or 4/6/2. The formula decides which global rank owns each task; the host placement decides which physical machine runs each rank. Therefore, if runtime and idle gap improve, the improvement comes from better process placement, not from a different scheduling algorithm.

3.3 Metric Collection

Each experiment records both correctness and timing metrics. Correctness is checked using the final people counts for each frame. Timing metrics are collected to explain where the runtime is spent.

The main recorded metrics are:

$$T_{wall}, \quad (3.9)$$

the total end-to-end runtime;

$$T_{compute,r}, \quad (3.10)$$

the YOLO inference and local processing time of rank r ;

$$T_{comm,r}, \quad (3.11)$$

the time rank r spends sending or receiving results; and

$$T_{no_comm} = T_{wall} - T_{comm}. \quad (3.12)$$

These metrics are necessary because speedup alone is not enough. A run can have many parallel tasks but still be slow if rank 0 spends too much time gathering detections, or if a slow machine receives too many tasks.

3.4 Load Balancing and Metrics

For each rank r , the total rank time is modeled as:

$$T_r = T_{compute,r} + T_{comm,r} + T_{idle,r}. \quad (3.13)$$

The idle-gap indicator is:

$$idle_gap = \frac{T_{max} - T_{min}}{T_{max}}, \quad (3.14)$$

where $T_{max} = \max_r T_r$ and $T_{min} = \min_r T_r$. A smaller idle gap means the workload is distributed more evenly.

4 Experimental Setup

4.1 Physical Cluster

The project uses three physical MacBook machines connected through a local network. The benchmark mode uses CPU execution to match the course emphasis on MPI processes and processor assignment. GPU/MPS execution is kept for demonstration and live-camera use rather than formal CPU speedup measurement.

Table 4.1: Physical cluster roles

Host	Machine type	Role in experiments
master	MacBook Air M4	Coordinates execution, gathers results, stores outputs, and can run local tasks.
node1	MacBook Pro M4	Worker host; measured results suggest it can sustain more assigned ranks.
node2	MacBook Air M2	Worker host; weighted placement assigns fewer ranks to reduce imbalance.

4.2 Software and Data

The implementation uses C++17, OpenMPI [5, 3], Python helper scripts, and a pretrained Ultralytics YOLO detector [7]. The main dataset source is MOT17-derived video data [4]. Compact MOT17 subsets are used for quick correctness and granularity experiments; longer sequence segments are used for the required input-size and speedup runs.

Table 4.2: Main YOLO experimental configuration

Item	Setting
Detector	Pretrained YOLO11n, person-class filtering
Parallel framework	C++17 and OpenMPI
Benchmark device	CPU
Dataset	MOT17-derived sequences
Main process count	12 MPI processes
Main long-run grid	5x4 regions
Scheduler	Static block-cyclic task assignment
Placement variants	Uniform 4/4/4 and weighted 3/6/3, 4/6/2

4.3 Metrics

The report uses the following metrics from the project reports and generated CSV design:

- serial-versus-MPI correctness: mismatched frames and count error;
- detector accuracy: MAE, RMSE, percentage error, exact-match ratio;
- runtime with communication: wall-clock time including coordination;
- runtime without communication: compute-oriented time reported by the scripts;
- speedup and efficiency: $S_p = T_1/T_p$, $E_p = S_p/p$;
- idle-gap indicator for load-balance analysis.

5 Results and Discussion

5.1 Detector Accuracy Under Tile Splitting

This experiment checks whether splitting each frame into spatial tiles changes YOLO detector accuracy against MOT17 ground-truth counts. It evaluates detector behavior under tiling, not parallel speedup or latency.

Table 5.1: YOLO accuracy comparison across baseline and tile-splitting configurations

Configuration	Grid	Frames	Runtime (s)	MAE	RMSE	Percent err.	Pred. avg.
Single-machine baseline	1x1	300	86.4	7.983	8.269	0.490	8.223
MPI tiled	2x2	300	31.0	7.983	8.269	0.490	8.223
MPI tiled	4x3	300	37.0	8.223	8.517	0.504	7.977
MPI tiled	5x4	300	54.764	8.383	8.682	0.514	7.812

Table 5.1 reports the official 300-frame runtimes for the baseline and tile-splitting configurations. The 1x1 baseline is not tile-parallel, so it cannot use the extra MPI work units created by spatial splitting and is slower than the 4x3 MPI tiled run despite having fewer detector calls. Among the MPI tiled configurations, runtime generally increases from 2x2 to 4x3 to 5x4 because each frame creates more tile tasks and rank 0 must merge more intermediate detections. This increase is not linear in the number of tiles: 2x2 uses fewer tasks, but each tile has a larger resolution and therefore the per-tile inference cost remains significant. The 2x2 configuration has the same MAE, RMSE, percent error, and predicted average count as the single-machine baseline, meaning moderate tiling preserves detector behavior on this subset. Finer grids slightly degrade accuracy: MAE rises to 8.223 for 4x3 and 8.383 for 5x4, while the predicted average count falls from 8.223 to 7.812. This suggests that aggressive tiling introduces border effects, reduced image context, and imperfect duplicate handling near tile boundaries.

5.2 Input-Size Selection

This experiment chooses the main workload size required by the course. Here, “Frames” means the number of video frames processed in one run, not the number of videos. The table should be read as input-size selection rather than a perfectly controlled scaling experiment, because the compact quick tests use a 4x3 grid while the longer runs use the final 5x4 grid.

Table 5.2: YOLO input-size selection

Frames	P	Grid	Tasks	Runtime with comm. (s)	Runtime without comm. (s)	ms/task	Selection note
30	12	4x3	360	9.165	2.999	25.46	Compact quick test
60	12	4x3	720	12.570	6.801	17.46	Compact quick test
100	12	4x3	1200	17.776	11.596	14.81	Compact quick test
150	12	4x3	1800	23.585	17.103	13.10	Compact quick test
300	12	5x4	6000	54.764	49.156	9.13	Long sequence
600	12	5x4	12000	123.667	114.352	8.97	Selected N
1200	12	5x4	24000	129.852	128.122	5.41	$2N$, weighted run

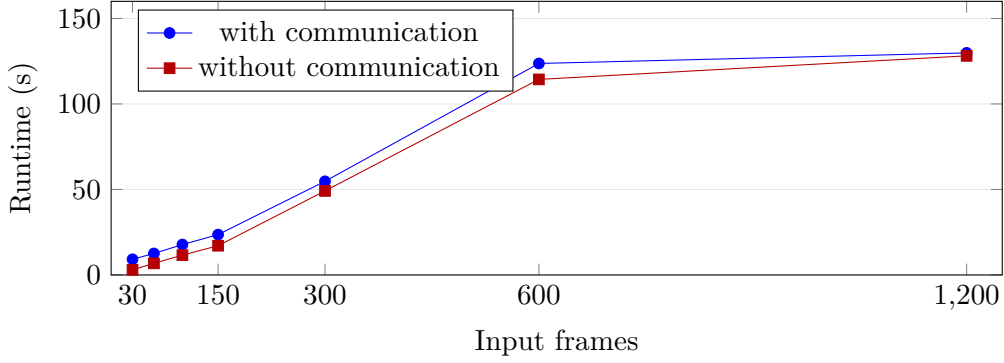


Figure 5.1: Runtime versus input size. The 600-frame workload is the first measured case inside the required 120–180 second interval.

The time-per-task column shows that very small inputs have high overhead because startup, synchronization, and aggregation are amortized over too few tasks. The selected workload is $N = 600$ frames because it is the first long-sequence case whose 12-process wall-clock runtime, 123.667 seconds, satisfies the required 120–180 second range. The 1200-frame row is used as the $2N$ speedup workload and uses weighted placement, so it should not be compared naively against the 600-frame row as if only the frame count changed.

5.3 Granularity and Load Balance

This experiment asks whether finer tile granularity creates a more balanced workload and how much communication it adds. The weighted placement keeps the idle-gap indicator at 0.235, within the 25 percent course criterion, but increasing the number of regions still increases task count and communication.

Table 5.3: YOLO granularity and load balance with weighted 6/4/2 placement

Grid	P	Tasks	Task/rank	Max comp.	Comm.	Comm/task	Idle gap	Balance
1x1	12	150	12.5	0.731	10.674	71.16	0.235	Balanced
2x2	12	600	50.0	3.155	16.306	27.18	0.235	Balanced
4x3	12	1800	150.0	9.266	34.881	19.38	0.235	Balanced
5x4	12	3000	250.0	13.733	51.790	17.26	0.235	Balanced

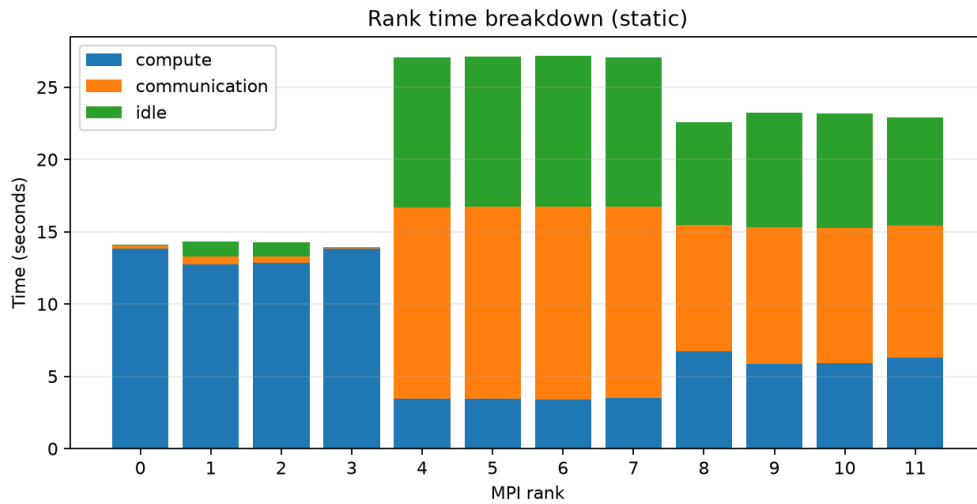


Figure 5.2: Per-process timing breakdown for the granularity experiment. Each bar is one MPI process, with compute, communication, and idle time stacked in the same column.

Increasing the tile grid increases the number of frame-tile tasks from 150 at 1x1 to 3000 at

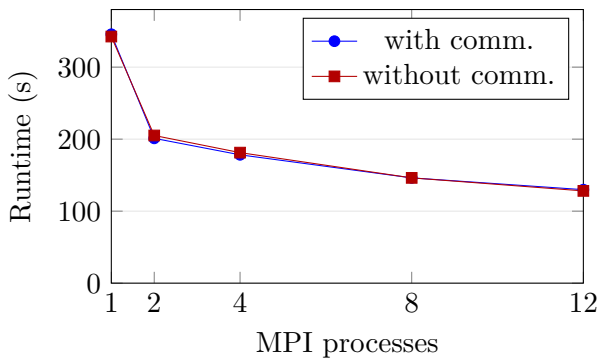
5x4. This gives the scheduler more work units, but total communication also rises from 10.674 seconds to 51.790 seconds and the master must merge more intermediate detections. The idle-gap indicator remains 0.235 under the weighted placement, so the result supports two claims: weighted placement can keep the workload balanced, but finer granularity is not automatically faster because communication and post-processing overhead also increase. Figure 5.2 shows why the load-balance check must inspect process-level timing rather than only total runtime.

5.4 Speedup and Efficiency

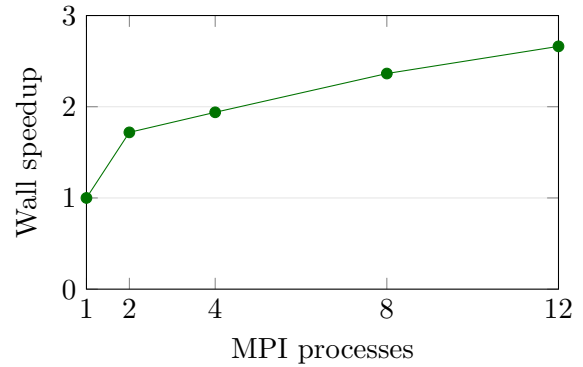
The speedup experiment uses the selected input size doubled to $2N = 1200$ frames and applies weighted static placement for the multi-rank runs. This keeps the process distribution consistent with the heterogeneous cluster: stronger machines receive more ranks, while the slower machine receives fewer ranks.

Table 5.4: YOLO speedup on 1200 frames with weighted placement

P	Runtime with comm. (s)	Runtime without comm. (s)	Wall speedup	Efficiency	Note
1	345.677	342.467	1.000	1.000	Baseline
2	201.111	205.100	1.719	0.859	Weighted placement
4	178.238	181.289	1.939	0.485	Weighted placement
8	146.212	146.204	2.364	0.296	Weighted placement
12	129.852	128.122	2.662	0.222	Weighted 4/6/2



(a) Runtime by process count



(b) Wall-clock speedup

Figure 5.3: Runtime and speedup trends for the 1200-frame weighted-placement run.

The speedup is measurable but sublinear. Wall-clock speedup rises from 1.719x at two processes to 2.662x at twelve processes, showing that the MPI pipeline benefits from distributing frame-tile inference across machines. However, efficiency drops as P increases, so each added process contributes less marginal benefit because of communication, aggregation at rank 0, heterogeneous node speeds, and duplicated work around tile boundaries.

The parallel cost confirms the same trend. Using the standard cost model $C_p = pT_p$, with T_p taken from the runtime including communication, the two-process run costs $2 \times 201.111 = 402.222$ process-seconds, while the twelve-process run costs $12 \times 129.852 = 1558.224$ process-seconds. Compared with the serial cost $C_1 = 345.677$, the overhead $O_p = C_p - C_1$ grows from 56.545 process-seconds at $P = 2$ to 1212.547 process-seconds at $P = 12$.

Table 5.5: Parallel cost for the 1200-frame speedup experiment

P	Runtime T_p (s)	Cost $C_p = pT_p$	Overhead $O_p = C_p - C_1$
1	345.677	345.677	0.000
2	201.111	402.222	56.545
4	178.238	712.952	367.275
8	146.212	1169.696	824.019
12	129.852	1558.224	1212.547

Therefore the implementation is useful for reducing wall-clock time, but it is not cost-optimal at high process counts: the cluster spends much more total process time to obtain a smaller elapsed time. In this workload, $P = 12$ is justified by the course requirement and by the 2.662x wall-clock speedup, while the cost analysis explains why efficiency falls to 0.222.

5.5 Weighted Static Placement

This experiment tests whether equal rank placement is appropriate on the heterogeneous three-MacBook cluster. The best measured weighted placement assigns four ranks to the master, six to node1, and two to node2, giving more ranks to the stronger machine and fewer ranks to the slower one.

Table 5.6: Weighted static placement on heterogeneous cluster

Placement	Distribution	Runtime with comm. (s)	Runtime without comm. (s)	Idle gap	Balance
Uniform	4/4/4	40.619	36.522	0.466	Not balanced
Weighted	3/6/3	34.712	30.339	0.371	Not balanced
Weighted	4/6/2	29.581	27.484	0.235	Balanced

Weighted placement is the strongest load-balancing result in the report. Uniform 4/4/4 placement has a higher idle gap and fails the 0.25 balance criterion. Moving to weighted placements improves runtime and idle gap because stronger machines receive more MPI ranks while the slower machine receives fewer ranks. The best measured placement is 4/6/2, which reduces the idle gap below the balance threshold. This supports the claim that processor assignment matters: equal rank counts are not necessarily fair when hosts have different throughput.

5.6 Non-Blocking Gather Evaluation

The final optimization branch keeps the same static task mapping and YOLO inference logic, but replaces the blocking result collection with a two-phase non-blocking gather. The purpose of this experiment is to check whether the communication change preserves correctness and improves the end-to-end runtime on the 12-rank CPU cluster.

Table 5.7: Summary of the non-blocking gather branch

Measurement	Main result	Interpretation
Correctness	Passed	Serial and MPI outputs match, so non-blocking collection does not change bounding boxes or final counts.
Detector accuracy	MAE 8.35, RMSE 8.68	YOLO undercounts crowded MOT17 frames: ground truth averages 16.21 people while prediction averages 7.86. This is a detector limitation, not an MPI error.
Input-size run	600 frames, 35.45 s	The optimized gather reduces end-to-end runtime compared with the earlier conservative benchmark.
Granularity	5x4 grid, idle gap 0.540	Fine tiling creates many payloads and more merge work. Non-blocking transfer reduces communication waiting, but it does not remove natural load imbalance in video inference.
Speedup	3.342x at $P = 12$	Runtime drops from 264.18 s at $P = 1$ to 79.06 s at $P = 12$, with efficiency about 0.28.

The non-blocking speedup trend is stronger than the earlier blocking-oriented benchmark: $T_1 = 264.18$ s, $T_2 = 230.92$ s, $T_4 = 227.60$ s, $T_8 = 121.85$ s, and $T_{12} = 79.06$ s. The largest jump happens between 4 and 8 ranks, where the workload begins to benefit more clearly from spreading inference across machines. The speedup is still sublinear because rank 0 must gather all detections and run global duplicate removal, and because the serial merge stage limits scaling according to Amdahl’s law.

In a few runs, the measured runtime without communication is slightly larger than the corresponding runtime with communication, especially at $P = 2$ and $P = 4$. This does not mean communication has negative cost. These measurements were collected from separate runs, so small inversions can come from system noise, background operating-system activity, cache state, and CPU scheduling variation. The report therefore treats these small differences as measurement noise rather than a change in the metric definition.

5.7 YOLO Parallel Correctness

This experiment verifies parallel correctness: after task assignment, MPI gathering, coordinate remapping, duplicate removal, and final counting, the MPI output should match the serial pipeline output.

Table 5.8: YOLO serial-versus-MPI correctness

Result	Schedule	Grid	P	Frames	Mismatch	Max err.	Mean err.
Passed	Static	4x3	12	30	0	0	0.000

The MPI run has zero mismatched frames over the 30-frame verification sequence. Both the maximum count error and mean count error are zero, so the parallel pipeline exactly reproduces the serial pipeline for this tested configuration. This proves serial-versus-MPI consistency; it does not prove that YOLO is accurate against human ground truth, which is measured separately in the detector accuracy section.

5.8 Summary of Findings

The measured YOLO results support the main task-parallel narrative. The implementation preserves serial output, reaches the required input-size runtime, shows measurable speedup, and demonstrates that task granularity and rank placement strongly affect performance. The strongest measured improvement in load balance comes from weighted static placement, which reduces the idle-gap indicator from 0.466 to 0.235.

6 Conclusion

This project implemented and evaluated a task-parallel people-counting pipeline using YOLO, C++17, and OpenMPI on a three-machine physical cluster. The input video is decomposed both temporally and spatially: frames create independent work over time, while image tiles create finer-grained work inside each frame. These tasks are assigned to MPI ranks using static block-cyclic mapping, processed locally by YOLO workers, and merged at the master process through coordinate remapping, tile-owner filtering, and global duplicate removal.

The experiments show that the parallel version preserves the behavior of the serial tiled pipeline while exposing the main performance trade-offs of distributed video inference. Task granularity affects both parallelism and communication cost. Static scheduling keeps the implementation simple and low-overhead, but it must be combined with careful process placement on a heterogeneous cluster. Weighted placement improves the final configuration by reducing idle time without changing the algorithmic output. The final non-blocking gather branch further improves result collection by allowing payload messages from different ranks to be in flight at the same time, while keeping YOLO inference and the merge logic unchanged.

The project also shows the practical limitations of this approach. Centralized merging at the master can become a bottleneck, tiled inference may reduce visual context near region boundaries, and detector accuracy on crowded scenes is limited by the pretrained model rather than by MPI itself. Even with these limitations, the system demonstrates the key parallel programming concepts required by the course: decomposition, mapping, communication, load balancing, correctness verification, granularity analysis, and speedup measurement. The live-camera mode further illustrates that the same MPI pipeline can be used as an interactive distributed computer-vision application.

Bibliography

- [1] Marco Farrugia. Yolo11 architecture diagram: Visual representation of the yolo11 object detection model. <https://doi.org/10.5281/zenodo.17508018>, 2025.
- [2] Keymeulen. Crowd crossing the street, hungary 2011. https://commons.wikimedia.org/wiki/File:Crowd_crossing_the_street,_Hungary_2011.jpg, 2011. CC0 1.0 Universal Public Domain Dedication.
- [3] Message Passing Interface Forum. Mpi: A message-passing interface standard. <https://www.mpi-forum.org/docs/>, 2023.
- [4] MOTChallenge. Mot17 benchmark. <https://motchallenge.net/data/MOT17/>, 2026.
- [5] Open MPI Project. Open mpi documentation. <https://docs.open-mpi.org/>, 2026.
- [6] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 779–788, 2016.
- [7] Ultralytics. Ultralytics yolo documentation. <https://docs.ultralytics.com/>, 2026.